Ders 10 Genişletilmiş Notu: Aşağı İtmeli Otomatlar: Yığın, Geçişler, Anlık Betimleme, Örnekler ve Biçimsel Tanım

Ders 10 Genişletilmiş Notu: Aşağı İtmeli Otomatlar: Yığın, Geçişler, Anlık Betimleme, Örnekler ve Biçimsel Tanım

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Bağlamdan Bağımsız Gramerlerden Aşağı İtmeli Otomatlara
- 2. Aşağı İtmeli Otomatın Yapısı
- 3. Geçişler: Okuma, Çekme, İtme
- 4. Deterministik Olmama ve Makinenin Çoğalması
- 5. Birinci Örnek: $a^n b^n$ Dilini Tanıyan PDA
- 6. PDA Bir Dizgiyi Ne Zaman Kabul Eder?
- 7. İkinci Örnek: vv^R Dilini Tanıyan PDA
- 8. Kısa Gösterim: Dizgi Çekme ve Dizgi İtme
- 9. Üçüncü Örnek: a ve b Sayısı Eşit Olan Dizgiler
- 10. Biçimsel Tanımlar
- 11. Bağlamdan Bağımsız Diller ve PDA'ların Denkliği

1. Bağlamdan Bağımsız Gramerlerden Aşağı İtmeli Otomatlara

Geçen derste bağlamdan bağımsız gramerler (context-free grammar, CFG) işlenmişti. Bu derste aşağı itmeli otomat (pushdown automaton) denen yapı görülür. Kısaca PDA diye yazılır ve hoca ders boyunca bu kısaltmayı kullanır. Dersin başında söylenen temel sonuç şudur: PDA'lar bağlamdan bağımsız gramerlerle eşdeğerdir. Bu denklik dersin sonunda teorem olarak yazılır, ispatı sonraki derse kalır.

2. Aşağı İtmeli Otomatın Yapısı

2.1 Bildiğimiz Parçalar

PDA, şimdiye kadar gördüğümüz otomatlara çok benzer. Durumları (state) ve durumlar arasında geçişleri (transition) vardır. Durumların bazıları kabul durumu (accept state), bazıları ret durumudur (reject state). Bir de başlangıç durumu (start state) vardır. Otomatın okuduğu bir giriş şeridi (input tape), daha doğrusu bir giriş dizgisi (input string) bulunur.

2.2 Yeni Parça: Yığın

PDA'da yeni olan şey yığındır (stack). Yığın, üstüne bir şey koyduğumuz (push) ve yine üstten bir şey aldığımız (pop) veri yapısıdır. Son giren ilk çıkar. Bu yapıya LIFO (last in, first out) denir.

Giriş dizgisi üzerindeki okuma-yazma kafasına ek olarak bir de yığın kafası olduğunu düşünebiliriz. Yığın kafası her zaman en son eklenen sembolü, yani yığının tepesini gösterir.

2.3 Yığının Dibindeki Özel Sembol

Sıfırıncı anda, yani hesaplama başlamadan önce, yığında genellikle yalnızca özel bir sembol vardır. Bu derste bu sembol olarak $ kullanılır. Sembolün $ olması şart değildir. PDA'ları öyle yazarız ki yığının tepesinde $ gördüğünde makine yığının sonuna geldiğini anlar.

3. Geçişler: Okuma, Çekme, İtme

3.1 Bir Geçişin Üç Parçası

DFA gibi önceki otomatlarda geçişin üzerine yalnızca a ya da b yazıyorduk. Bunun anlamı "girişte a gördüysek q_2'ye geçebiliriz" idi. PDA'da yığının içeriğini de düşünmek gerekir. q_1'den q_2'ye giden bir geçiş şöyle yazılır:

$$q_1 \xrightarrow{a,\ b \to c} q_2$$

Bu geçişin olabilmesi için girişte sıradaki sembol a, yığının tepesindeki sembol de b olmalıdır. Geçiş olunca üç şey gerçekleşir:

- Girişte bir sonraki sembole geçilir, yani a okunmuş olur.
- Yığının tepesindeki b çekilir.
- Yığına c itilir ve b'nin yerinde artık c durur.

Demek ki bir geçişin artık üç parçası vardır: giriş sembolü (input symbol), çekilen sembol (pop symbol) ve itilen sembol (push symbol).

3.2 Null Kullanımı

Her geçişte bir şey okumak, çekmek ya da itmek zorunda değiliz. Bunu null ile, yani ε ile gösteririz. Üç yerin herhangi birine null yazılabilir:

- **Giriş yerinde null:** Geçiş, girişten hiçbir şey okumadan olur.
- **Çekme yerinde null:** Yığından hiçbir şey çekilmez.
- **İtme yerinde null:** Yığına hiçbir şey itilmez.

Null, üç yerin birinde, ikisinde ya da üçünde birden bulunabilir. Hocanın örneği q_1'den çıkan $a, \varepsilon \to \varepsilon$ geçişidir. Bu geçiş girişten a okur, yığından çekmez, yığına itmez. Yığında tepeden $B, A, \$$ varsa geçiş olduktan sonra yığın yine $B, A, \$$ olur, yani yığında hiçbir şey değişmez.

3.3 Boş Yığından Çekmeye Çalışmak

Yığın tamamen boşken q_1'den çıkan bir geçiş yığından b çekmek istesin. Yığında bir şey olmadığına göre ne olur? Bu durumda PDA durur (halt) ve girişi reddeder.

> **Hocanın vurgusu:** PDA'larımızı bu durum hiç oluşmayacak biçimde tasarlarız. Yığının dibinde genellikle $ bulunur ve $'ı yalnızca hesaplamaların sonunda kullanmaya dikkat ederiz.

4. Deterministik Olmama ve Makinenin Çoğalması

4.1 İki Kaynak

Giriş sembolünün null olabileceğini söylemek bize deterministik olmamayı (non-determinism) hatırlatır. Bu derste PDA'lar deterministik değildir. Hoca deterministik olmamanın iki kaynağını hatırlatır:

- **Aynı parametrelerle farklı durumlara gitmek:** q_1'den $a, b \to c$ ile q_2'ye giden bir geçiş varken, aynı $a, b \to c$ parametreleriyle q_3'e giden bir geçiş de olabilir.
- **Girişten okumadan geçiş yapmak:** q_1'den q_2'ye giriş kısmı ε olan bir geçiş olabilir.

Ders sonrası not: Hoca, giriş sembolünün null olabilmesini makinenin deterministik olmadığının göstergesi olarak söyledi. Doğrusu: ε-geçişi tek başına deterministik olmamayı gerektirmez. Belirleyici olan, aynı anda birden fazla geçişin uygulanabilmesidir. Girişten okumayan bir geçiş, aynı durumda ve aynı yığın tepesinde başka bir geçişle çakışmıyorsa makine yine deterministik kalabilir. Bu dersteki tanımda bu tür çakışmalara izin verildiği için PDA'lar deterministik değildir.

Ek bilgi (derste anlatılmadı): DFA ile NFA aynı dilleri tanır, ama PDA'larda durum farklıdır. Deterministik PDA'ların tanıdığı diller, bütün bağlamdan bağımsız dillerin yalnız bir alt kümesidir. Bölüm 7'deki $\{vv^R\}$ dili bunun bir örneğidir. Bu dili deterministik bir PDA tanıyamaz.

4.2 Makine Kendini Çoğaltır

Deterministik olmayan bir geçişte hesaplama makinesi kendini çoğaltır (replicate). Sanki artık paralel çalışan iki hesaplama vardır. Hoca bunu küçük bir makineyle gösterir. Makinenin bir geçişi $a, b \to c$ biçimindedir. q_1'den q_2'ye giden başka bir geçiş ise girişten okumaz, yığından çekmez, yalnızca yığına C iter ($\varepsilon, \varepsilon \to C$).

Giriş $abcacc$ olsun ve ilk sembol okunmuş olsun. Makine q_1 durumunda, okunmamış giriş $bcacc$, yığın içeriği ise tepeden $A, \$$ olsun. Bu üç bilgi, makinenin o anki durumunu belirleyen bütün temel bilgidir ve şöyle yazılır:

$$(q_1,\ bcacc,\ A\$)$$

Bu yazıma anlık betimleme (instantaneous description, ID) denir. Ayrıntısına Bölüm 5 ve 10'da dönülür.

Şimdi $\varepsilon, \varepsilon \to C$ geçişi olunca iki makine oluşur. Yeni makine q_2 durumundadır. Girişten okumadığı için kalan giriş değişmez. Yığından çekmez, yalnızca C iter. Dolayısıyla iki anlık betimleme vardır:

- **1. makine:** $(q_1,\ bcacc,\ A\$)$
- **2. makine:** $(q_2,\ bcacc,\ CA\$)$

Bu makinelerin her biri paralel olarak çalışmaya devam eder.

4.3 Paralel Makinelerde Kabul

Paralel makinelerden biri girişi kabul ederse, özgün makine ya da kopyalarından hangisi olduğu fark etmeksizin, PDA girişi kabul eder.

5. Birinci Örnek: $a^n b^n$ Dilini Tanıyan PDA

5.1 Makine

Dili $L(M) = \{a^n b^n \mid n \geq 0\}$ olan bir makineye bakalım. Dört durumu vardır: q_0 başlangıç ve kabul durumu, q_3 son kabul durumu, q_1 ile q_2 ise kabul durumu olmayan durumlardır. Hoca dersin sonuna doğru bu makineye M_1 adını verir.

Geçiş	Etiket	Görevi
$q_0 \to q_1$	$\varepsilon, \varepsilon \to \varepsilon$	Her şeyi null olan geçiş
$q_1 \to q_1$	$a, \varepsilon \to a$	a gördükçe yığına a it
$q_1 \to q_2$	$b, a \to \varepsilon$	İlk b ile eşleştirmeye başla
$q_2 \to q_2$	$b, a \to \varepsilon$	Her b için bir a çek
$q_2 \to q_3$	$\varepsilon, \$ \to \$$	Yığının sonuna gelindi mi diye denetle

Döngüden nasıl çıkılır? $q_2 \to q_3$ geçişinde girişten bir şey okumak gerekmez. Yığının tepesinde $\$$ varsa yığının sonundayız demektir. Bu geçiş bir denetim gibidir: Yığının sonuna geldik mi? Geldiysek q_3'e geçeriz. O noktada giriş de bitmişse kabul ederiz.

> **Ders sonrası not:** Hoca q_0'ın kabul durumu olmasının sebebini söylemedi. Sebebi $n = 0$ durumudur. Boş dizgide makine q_3'e ulaşamaz, çünkü q_2'ye geçmek için bir b okumak gerekir. Simülasyonda q_0 kabul durumu olmaktan çıkarılınca makine yalnız boş dizgiyi yanlış reddetti.

Döngülerin görevi bellidir. q_1'deki döngü a'ları yığına iter. $q_1 \to q_2$ geçişiyle başlayan kısım ise b'leri yığındaki a'larla eşleştirir. Girişin sonunda yığının tepesinde \$ varsa, yığında başka bir şey kalmamış demektir. Bu da a sayısının b sayısına eşit olduğunu gösterir: Bütün a'lar ve b'ler eşleşmiştir.

5.2 PDA İki Değeri Eşleştirir

Hoca burada bir gözlem yapar. Bu PDA iki değeri, yani a'ların ve b'lerin sayısını eşleştirir. Üstelik yalnız sayıları değil, ardışık gelmelerini de ister. Hoca tahtaya "PDA'lar iki değeri eşleştirebilir, üç ya da daha fazla değeri eşleştiremez" diye yazar, sonra bu cümleden emin olmadığını söyleyerek siler. Yine de genel olarak yalnız iki değerin eşleştirilebileceğini düşündüğünü, sebebinin de PDA'ların bağlamdan bağımsız gramerlere denk olması ve bu gramerlerin yalnız iki şeyi eşleştirebilmesi olduğunu ekler.

> **Ders sonrası not:** Hoca PDA'ların genel olarak yalnız iki değeri eşleştirebileceğini söyledi. Doğrusu: Bu cümle kaba hâliyle doğru değildir. $\{a^n b^n c^m d^m\}$ dili bağlamdan bağımsızdır ve iki ayrı çift eşleştirir. Bu dili tanıyan bir PDA, $\{a, b, c, d\}$ üzerinde 8 sembole kadar bütün dizgilerde simülasyonla da doğrulandı. Üç sayının birlikte eşit olmasını isteyen $\{a^n b^n c^n\}$ dili ise bağlamdan bağımsız değildir.

5.3 $aaabbb$ Girişinin Hesaplanması

Giriş $aaabbb$ olsun. Sıfırıncı anda ne yazarız? Anlık betimlemenin birinci parçası o anki durum, ikinci parçası girişin henüz okunmamış kısmı, üçüncü parçası yığın içeriğidir. Yığında başlangıçta yalnız özel sembol olduğundan betimleme $(q_0, aaabbb, \$)$ olur.

Makine çalışmaya başlayınca şunlar olur:

Zaman	Anlık betimleme	Kullanılan geçiş
0	$(q_0, aaabbb, \$)$	Başlangıç
1	$(q_1, aaabbb, \$)$	$\varepsilon, \varepsilon \to \varepsilon$
2	$(q_1, aabbb, a\$)$	$a, \varepsilon \to a$
3	$(q_1, abbb, aa\$)$	$a, \varepsilon \to a$
4	$(q_1, bbb, aaa\$)$	$a, \varepsilon \to a$
5	$(q_2, bb, aa\$)$	$b, a \to \varepsilon$
6	$(q_2, b, a\$)$	$b, a \to \varepsilon$
7	$(q_2, \varepsilon, \$)$	$b, a \to \varepsilon$
7'den sonraki kopya	$(q_3, \varepsilon, \$)$	$\varepsilon, \$ \to \$$

Adımlar şöyle okunur. Birinci adımda her şeyi null olan geçiş kullanılır. Durum q_1 olur, kalan giriş ve yığın aynı kalır. İkinci adımda sıradaki sembol a'dır. $a, \varepsilon \to a$ geçişi yığında ne olduğuna bakmaz, dolayısıyla uygulanabilir. Kullanılabilecek hiçbir geçiş kalmazsa makine durur. Burada durmaz, çünkü geçiş vardır. Durum yine q_1'dir, kalan giriş $aabbb$, yığın $a\$$ olur. Üçüncü ve dördüncü adımlarda aynı döngü devam eder ve a^n kısmı okunmuş olur.

> **Ders sonrası not:** Hoca tahtada 4. adımda yığına bir a daha yazmayı unuttu ($aa\$$ yazdı), 5. adıma geçerken fark edip düzeltti. Doğru betimleme, tablodaki $(q_1, bbb, aaa\$)$'dır.

Beşinci adımda değişiklik görülür. Girişte artık a yoktur ve $a, \varepsilon \to a$ geçişi uymaz. Ancak sıradaki sembol b olduğu için makine durmaz: b okunur, a çekilir, hiçbir şey itilmez ve durum q_2 olur. Bu noktada kalan giriş ile yığın içeriği arasındaki uyuma dikkat edin. Girişte kalan her b için yığında bir a vardır. Bu PDA bunun için tasarlanmıştır.

Altıncı ve yedinci adımlarda aynı şey sürer. Yedinci adımda girişte hiçbir şey kalmaz. Bunu kalan giriş yerine null yazarak gösteririz. Artık b okuyan geçiş kullanılamaz. Fakat $\varepsilon, \$ \to$

$ geçişi yüzünden makine kendini çoğaltır. Birinci makine $(q_2, \varepsilon, \$)$'de kalır. İkinci makine $(q_3, \varepsilon, \$)$'ye geçer. Giriş okunmaz, $ çekilip geri itilir.

Bu noktada başka geçiş kalmadığından iki makine de durur. Birinci makine q_2 kabul durumu olmadığı için reddeder. İkinci makine q_3 kabul durumu olduğu için kabul eder. Sonuç olarak PDA $aaabbb$ dizgisini kabul eder.

> **Ders sonrası not:** Hoca birinci adımı çoğalma olarak yazmadı. Bölüm 4'teki kurala göre q_0'daki makine de $(q_0, aaabbb, \$)$ olarak kalır, ama girişi tüketemeden durduğu için kabul edemez. Kabul sonucu değişmez.

6. PDA Bir Dizgiyi Ne Zaman Kabul Eder?

Aradan sonra hoca kabul koşulunu netleştirir. Bir PDA, şu iki koşulu sağlayan bir hesaplama (computation) varsa dizgiyi kabul eder:

- Girişin tamamı tüketilmiştir.
- Son durum bir kabul durumudur.

Yani paralel makinelerin ya da paralel hesaplamaların birinde hem bütün giriş okunmuş olmalı, hem de makine bir kabul durumunda bulunmalıdır. Yığının içeriği önemli değildir. Yığında a'lar, b'ler ya da başka semboller kalmış olabilir. Kabul durumundaysanız ve girişi bitirdiyseniz yeterlidir.

> **Hocanın vurgusu:** Bazı kaynaklarda yığının içeriği de önemsenir. Bu, okuduğunuz kitaba ve kaynağın kurallarına bağlıdır, o yüzden kaynağın kuralına dikkat edin. Bu dersin kuralında kabul eden hesaplamanın sonunda yığın içeriği önemsenmez.
>
> **Ek bilgi (derste anlatılmadı):** Son durumla kabul ve boş yığınla kabul, iki yaygın kuraldır. Bu iki kural aynı dil sınıfını verir. Bir kurala göre yazılmış PDA, aynı dili tanıyan ve öbür kurala göre çalışan bir PDA'ya dönüştürülebilir.

7. İkinci Örnek: vv^R Dilini Tanıyan PDA

7.1 Makine

İkinci makinenin dili şudur:

$$L(M) = \{vv^R \mid v \in \{a, b\}^*\}$$

Burada v, a ve b sembollerinin birleştirilmesiyle oluşan herhangi bir dizgi, v^R ise tersidir. Hoca bu makineye sonradan M_2 der. Üç durumu vardır: q_0 başlangıç durumu, q_1 ara durum, q_2 kabul durumu.

Geçiş	Etiket	Görevi
$q_0 \to q_0$	$a, \varepsilon \to a$ ve $b, \varepsilon \to b$	v kısmını yığına it
$q_0 \to q_1$	$\varepsilon, \varepsilon \to \varepsilon$	Hiçbir şey okumadan, çekmeden, itmeden geç
$q_1 \to q_1$	$a, a \to \varepsilon$ ve $b, b \to \varepsilon$	Okunanla çekileni eşleştir
$q_1 \to q_2$	$\varepsilon, \$ \to \$$	Yığının sonunu denetleyip kabule geç

Aynı durum üzerinde birden fazla geçiş varsa bunlar tek okun üzerine alt alta yazılabilir. Makine örneğin $aabbaa$ gibi bir dizgiyi kabul eder: $v = aab$ ise öbür tarafta tersi baa vardır.

q_0 döngüsünde okunan her şey yığına konur. Aynı zamanda her adımda makine kendini çoğaltır ve kopya yığından çekmeye başlar. Bu iki iş hep paralel yürür. Girişin sonunda yığının sonuna gelinip gelinmediğine bakılır: Yalnız $ kaldıysa kabul edilir. Bu yine bir eşleştirmedir.

Paralel düşünmek istemezseniz şöyle de düşünebilirsiniz: Birinci parça v kısmını yığına iter. İkinci parça yığından çeker ve girişteki v^R ile yığındaki v'nin eşleşip eşleşmediğine bakar. Hocanın kullandığı kaynağa göre makine, girişin ortasını deterministik olmayan biçimde tahmin eder.

7.2 $abba$ Girişinin Hesaplama Ağacı

Hoca makinenin nasıl çalıştığını açıkça göstermek için $abba$ girişinde bütün hesaplama ağacını (computation tree) çizer. Sıfırıncı anda tek bir makine vardır: $(q_0, abba, \$)$.

> **Ders sonrası not:** Hoca önce sıfırıncı anda da paralel hesaplama olduğunu söylemeye başladı, sonra vazgeçip ağacı birinci andan başlattı. Aslında ε-geçişleri sıfırıncı anda da uygulanabilir ve $(q_1, abba, \$)$ ile $(q_2, abba, \$)$ de oluşur. Bu iki kopya hemen ölür: q_1'de yığın tepesi $\$$ iken a okunamaz, q_2'den ise hiç geçiş çıkmaz. Simülasyonda ağacın kalanı ve kabul sonucu değişmedi.

1. zaman. q_0'daki makine a'yı okur ve yığına iter: $(q_0, bba, a\$)$. Aynı noktada $\varepsilon, \varepsilon \to \varepsilon$ geçişi de vardır, makine kendini çoğaltır: $(q_1, bba, a\$)$.

> **Ders sonrası not:** Hoca burada q_1'deki kopyadan $\varepsilon, \$ \to \$$ ile q_2'ye bir kopya daha çıkardı ve o anda üç durumun etkin olduğunu söyledi. Birkaç dakika sonra hatasını fark edip bu dalı sildi. Doğrusu: Bu geçiş yalnız yığının tepesi $\$$ iken uygulanabilir, oysa $(q_1, bba, a\$)$'de tepe a'dır.

2. zaman. Sıradaki sembol b'dir. q_0'daki makine b'yi okur ve iter: $(q_0, ba, ba\$)$. Bu makine de hemen çoğalır: $(q_1, ba, ba\$)$.

$(q_1, bba, a\$)$ ise ölür. $b, b \to \varepsilon$ geçişi için yığının tepesinde b olması gerekir, ama tepe a'dır. $\varepsilon, \$ \to \$$ geçişi için yığında yalnız $\$$ olmalıdır, o da yoktur. $a, a \to \varepsilon$ için ise girişte a gerekir. Geçişlerin hiçbiri uygulanamadığından bu hesaplama durur ve reddeder.

3. zaman. Üç makine vardır:

- q_0'daki makine b'yi okur ve iter: $(q_0, a, bba\$)$.
- Bu makineden hemen bir kopya çıkar: $(q_1, a, bba\$)$.
- $(q_1, ba, ba\$)$ makinesi b'yi okuyabilir, çünkü yığının tepesinde de b vardır: $(q_1, a, a\$)$. Yığının tepesi $\$$ olmadığı için buradan q_2'ye geçilemez.

4. zaman. $(q_0, a, bba\$)$ makinesi a'yı okur ve iter: $(q_0, \varepsilon, abba\$)$. Kopyası $(q_1, \varepsilon, abba\$)$ olur. $(q_1, a, bba\$)$ makinesi ölür, çünkü yığının tepesi b iken a okunamaz. $(q_1, a, a\$)$ makinesi a okur ve a çeker: $(q_1, \varepsilon, \$)$. Yığının tepesi artık $\$$ olduğu için bu makineden kopya çıkar: $(q_2, \varepsilon, \$)$.

Bütün ağaç şöyledir (kopya, ε-geçişiyle çoğalan makinedir):

```
t=0  (q0, abba, $)
t=1  (q0, bba, a$)
       kopya: (q1, bba, a$)            -> ölür
t=2  (q0, ba, ba$)
       kopya: (q1, ba, ba$)
t=3  (q0, a, bba$)
       kopya: (q1, a, bba$)            -> ölür
     (q1, ba, ba$) dalından: (q1, a, a$)
t=4  (q0, ε, abba$)                    -> 1. hesaplama
       kopya: (q1, ε, abba$)           -> 2. hesaplama
     (q1, a, a$) dalından: (q1, ε, $)  -> 3. hesaplama
       kopya: (q2, ε, $)               -> 4. hesaplama
```

Beşinci anda girişte bir şey kalmadığından dört hesaplamanın hepsi durur. Sonuçları şöyledir:

Hesaplama	Son betimleme	Sonuç
1	$(q_0, \varepsilon, abba\$)$	Ret, q_0 kabul durumu değil
2	$(q_1, \varepsilon, abba\$)$	Ret, q_1 kabul durumu değil
3	$(q_1, \varepsilon, \$)$	Ret, q_1 kabul durumu değil

Hesaplama	Son betimleme	Sonuç
4	$(q_2, \varepsilon, \$)$	Kabul

4. hesaplama kabul ettiği için M, $abba$'yı kabul eder. Hoca, ağaçtaki makineler paralel olduğu için zamanları tek tek numaralamak yerine aynı numarayı paralel dallara vermenin daha uygun olduğunu da söyler.

   Hocanın vurgusu: Bütün hesaplama ağacını her zaman oluşturmak zorunda değilsiniz. Uygulamada bütün paralel dalları düşünmez, gerçekten işe yarayacak olanı düşünürsünüz. Ağaç burada makinenin içeride nasıl çalıştığını açıkça göstermek için yazıldı ve anlamaya yardım eder.

8. Kısa Gösterim: Dizgi Çekme ve Dizgi İtme

Bazen geçişte tek sembol yerine dizgi çekmek ve dizgi itmek istenir. Bunun için bir kısa gösterim (shortcut notation) vardır: $a, w_1 \to w_2$ yazılabilir. Burada w_1 çekilecek dizgi, w_2 itilecek dizgidir. Hocanın örneği şudur:

$$q_1 \xrightarrow{a,\ eb \to cdf} q_2$$

Bu geçişte girişten a okunur. Yığının tepesinde eb ve altında başka şeyler bulunmalıdır. eb çekilir, yerine cdf itilir.

Bu yalnız bir kısaltmadır. Makinenin kendisi sembol sembol çalışır. Perde arkasında geçiş örneğin şöyle olabilir:

$$q_1 \xrightarrow{a,\ e\to\varepsilon} \cdot \xrightarrow{\varepsilon,\ b\to\varepsilon} \cdot \xrightarrow{\varepsilon,\ \varepsilon\to f} \cdot \xrightarrow{\varepsilon,\ \varepsilon\to d} \cdot \xrightarrow{\varepsilon,\ \varepsilon\to c} q_2$$

Ara durumların adı önemli değildir. Önce a okunur ve e çekilir. Sonra girişten okumadan b çekilir. e ve b gittikten sonra ekleme başlar. Peki hangisi önce eklenir, c mi f mi? Önce f eklenmelidir, çünkü sonunda c tepede kalmalıdır. Sırayla f, d ve c itilir ve q_2'ye varılır. Hoca açılımın böyle olabileceğini ama böyle olmak zorunda olmadığını söyler. Kısa gösterimde burada ne olduğu önemsenmez.

Bu açılım, tepeden e, b ile başlayan bir yığın üzerinde simülasyonla çalıştırıldı. Sonuçta yığının tepesi c, d, f oldu ve altındaki içerik değişmedi.

9. Üçüncü Örnek: a ve b Sayısı Eşit Olan Dizgiler

9.1 Makine

Kısa gösterimi kullanan bir PDA örneği olarak hoca şu makineyi verir. q_1'de altı geçişli bir döngü vardır. $q_1 \to q_2$ geçişi ise bilinen $\varepsilon, \$ \to \$$ geçişidir: Hiçbir şey okumadan yığının sonunda olup olmadığımıza bakar. Hoca bu makineye sonradan M_3 der.

Okunan	Yığın tepesi	Etiket	Yapılan iş
a	\$	$a, \$ \to 0\$$	\$'ın üstüne 0 koy
a	0	$a, 0 \to 00$	0'ı iki 0 ile değiştir
a	1	$a, 1 \to \varepsilon$	1'i çek, hiçbir şey itme
b	\$	$b, \$ \to 1\$$	\$'ın üstüne 1 koy
b	1	$b, 1 \to 11$	1'i iki 1 ile değiştir
b	0	$b, 0 \to \varepsilon$	0'ı çek, hiçbir şey itme

Birinci geçişte tek sembol değil, 0\$ dizgisi itilir. Bölüm 8'de konuşulan budur. Benzer biçimde tek bir 1 yerine birden fazla 1 konabilir.

Bu makinenin dili, a ve b sembollerinden oluşan ve a sayısı b sayısına eşit olan dizgilerdir. Önceki örnekte a ve b sayılarını eşleştirirken sıralarını da istiyorduk. Burada sıra önemsenmez.

Tablodan anlaşılan işleyiş şudur: Yığın, o ana kadar okunan dizgide hangi harfin kaç tane fazla olduğunu tutar. 0'lar fazla a'ları, 1'ler fazla b'leri sayar. Karşı harf okununca bir tane silinir.

> **Ders sonrası not:** Hoca $\$$'ın yalnız hesaplamanın sonunda kullanılması gerektiğini söyledi. Doğrusu: Bu kural anlam olarak yığının dibine dokunulmaması ve yığının hiç boşalmaması demektir. M_3'te $a, \$ \to 0\$$ ve $b, \$ \to 1\$$ hesaplamanın ortasında da $\$$'ı çeker, ama hemen geri iter. Simülasyonda üç makinenin hiçbirinde yığın boşalmadı.

Hoca örnek bir dizgi söylerken önce $abaaabbbb$ gibi bir dizgi yazar, "bu olmaz" diyerek düzeltir ve düzeltilmiş hâlinde dört a ve dört b olduğunu, bu yüzden makinenin kabul etmesi gerektiğini söyler.

9.2 Hocanın Eve Bıraktığı Giriş: $abbbaa$

Hoca makinenin ayrıntısına girmez ve $abbbaa$ gibi girişlerle evde denemenizi ister. Çözüm aşağıdadır (ara adım eklendi):

Adım	Anlık betimleme	Kullanılan geçiş
0	$(q_1, abbbaa, \$)$	Başlangıç
1	$(q_1, bbbaa, 0\$)$	$a, \$ \to 0\$$
2	$(q_1, bbaa, \$)$	$b, 0 \to \varepsilon$
3	$(q_1, baa, 1\$)$	$b, \$ \to 1\$$
4	$(q_1, aa, 11\$)$	$b, 1 \to 11$
5	$(q_1, a, 1\$)$	$a, 1 \to \varepsilon$
6	$(q_1, \varepsilon, \$)$	$a, 1 \to \varepsilon$
7	$(q_2, \varepsilon, \$)$	$\varepsilon, \$ \to \$$

Yığının tepesi $\$$ olan her noktada (0, 2 ve 6. adımlar) $\varepsilon, \$ \to \$$ ile bir kopya çıkar. 0 ve 2. adımlardaki kopyalar girişi tüketemeden ölür, çünkü q_2'den geçiş çıkmaz. 7. adımdaki kopya girişi bitirmiş olarak q_2'dedir, dolayısıyla M_3 $abbbaa$'yı kabul eder. Dizgide üç a ve üç b vardır. Sonuç simülasyonla doğrulandı.

10. Biçimsel Tanımlar

10.1 Geçiş Fonksiyonu Küme Döndürür

Bir durumdan, aynı (giriş, çekilen, itilen) parametreleriyle birden fazla duruma gidilebildiğini biliyoruz. q_1 aynı anda hem q_2'ye hem q_3'e geçebilir. Bu yüzden geçiş fonksiyonunun (transition function) sonuçları kümedir. Örneğin q_1'deyken girişte a okuyup yığının tepesinde b istiyorsak:

$$\delta(q_1, a, b) = \{(q_2, c),\ (q_3, d)\}$$

Bu, makinenin q_2'ye geçip c itebileceğini ya da q_3'e geçip d itebileceğini söyler. İtilen semboller aynı olmak zorunda değildir. Semboller yerine dizgiler de yazılabilir:

$$\delta(q_1, a, w_1) = \{(q_2, w_2),\ (q_3, w_3)\}$$

Yani a okunur, w_1 çekilir, sonra ya w_2 ya da w_3 itilir.

10.2 PDA'nın Biçimsel Tanımı

Bir PDA 7 elemanlı bir demettir (7-tuple):

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z, F)$$

- Q: Durumlar.
- Σ: Giriş alfabesi (input alphabet), girişte bulunabilecek semboller.
- Γ: Yığın alfabesi (stack alphabet).
- δ: Her zamanki gibi geçiş fonksiyonu.
- q_0: Başlangıç durumu.
- Z: Yığın başlangıç sembolü (stack start symbol). PDA'lara özgüdür.
- F: Kabul durumları.

Giriş alfabesi ile yığın alfabesi ayrıdır. Girişte a okuyup yığına örneğin X koyabilirsiniz, a koymak zorunda değilsiniz.

Hoca Sipser'ın kitabında yığın başlangıç sembolünün olmadığını, tanımın 6'lı olduğunu ve bunun önemli olmadığını söyler.

> **Ek bilgi (derste anlatılmadı):** $ yığına konduğu için Γ'nın bir elemanıdır, yani $Z \in \Gamma$'dır. Sipser'da büyük olasılıkla $, makinenin ilk geçişinde ($\varepsilon, \varepsilon \to \$$) yığına itilir. Bu derste ise $ en baştan yığındadır.

10.3 Anlık Betimleme

Anlık betimleme üç şeyden oluşur:

$$(q, u, s)$$

Burada q durum, u kalan giriş, s o anki yığın içeriğidir. Örneğin $(q_1, aab, ab\$)$ şunu söyler: q_1 durumundayız. Girişte okunmamış aab vardır. Yığında tepeden $a, b, \$$ bulunur.

10.4 Hesaplama ve $\vdash$ Gösterimi

Anlık betimlemelerle hesaplama yazılabilir. Hesaplama, anlık betimlemelerin bir listesidir. Örneğin:

$$(q_0, aaabbb, \$) \vdash (q_1, aaabbb, a\$) \vdash \cdots \vdash (q_3, \varepsilon, \$)$$

Hoca $\vdash$ sembolünü "yields" diye okuduğunu ama emin olmadığını söyler ve bilen varsa söylemesini ister.

> **Ek bilgi (derste anlatılmadı):** Bu sembolün adı büyük olasılıkla turnstile'dır ve "yields" okunuşu yaygındır. Türkçede "verir" diye okunabilir.
>
> **Ders sonrası not:** Hoca bu satırı genel bir örnek olarak ("maybe") yazdı, M_1'in izlemesi olarak okunmamalıdır. M_1'de $q_0 \to q_1$ geçişi yığına bir şey itmez. Simülasyonda $(q_1, aaabbb, a\$)$ betimlemesine ulaşılmadı. M_1 için doğru dizi Bölüm 5.3'teki tablodur.

Buradaki null, işlenecek girişin kalmadığını gösterir. Başarılı bir hesaplama genellikle böyle görünür. Dizgiyi kabul etmek için girişin bitmesi gerekir. Yığının da $'a kadar temizlenmiş olması sık görülür ama zorunlu değildir.

> **Hocanın vurgusu:** Yığın kısmının böyle olması şart değildir, ama giriş kısmının null olması şarttır.

Bütün adımları yazmak yerine kısaltma kullanılabilir. $\vdash^*$ gösteriminde yıldız, araya sıfır ya da daha fazla adım alındığını belirtir:

$$(q_0, aaabbb, \$) \vdash^* (q_3, \varepsilon, \$)$$

10.5 PDA'nın Dili

Bu gösterimle bir PDA'nın dili tanımlanabilir. M makinesinin dili şu dizgilerin kümesidir:

$$L(M) = \{w \mid (q_0, w, Z) \vdash^* (q_f, \varepsilon, s),\ q_f \in F\}$$

Başlangıçta ilk durumdayız, girişte w'nin tamamı var ve yığında yalnız yığın başlangıç sembolü bulunuyor. Sıfır ya da daha fazla adım sonunda bir kabul durumuna varıyoruz. Dizginin hepsi tüketildiği için ikinci bileşen null'dur. Yığın içeriği s ise herhangi bir şey olabilir.

> **Hocanın vurgusu:** Yığında ne olduğu önemsenmez, ama ikinci bileşenin null olması önemlidir.

10.6 Örneklerin Dili

Hoca önceki örneklere ad verir: $a^n b^n$ makinesi M_1, vv^R makinesi M_2, sayı eşitleyen makine M_3.

M_1'de $(q_0, aaabbb, \$) \vdash^* (q_3, \varepsilon, \$)$ olduğundan $aaabbb \in L(M_1)$'dir. Genel olarak hoca şöyle yazar: $(q_0, a^n b^n, \$) \vdash^* (q_3, \varepsilon, \$)$ olduğu için $a^n b^n$ dizgileri M_1'in dilindedir. Bu kalıptaki bütün dizgiler bu koşulu sağladığından $L(M_1) = \{a^n b^n \mid n \geq 0\}$ denir.

> **Ders sonrası not:** Hoca $(q_0, a^n b^n, \$) \vdash^* (q_3, \varepsilon, \$)$ bağıntısını $n \geq 0$ için söyledi. Doğrusu: Bu bağıntı $n \geq 1$ için geçerlidir. $n = 0$ iken q_3'e ulaşılamaz. Boş dizgi, sıfır adımda $(q_0, \varepsilon, \$)$'de kalıp q_0 kabul durumu olduğu için kabul edilir.

> **Ders sonrası not:** Hoca bu bağıntıdan $L(M_1)$'in kalıba eşit olduğunu çıkardı. Doğrusu: Bu bağıntı yalnız $a^n b^n$ dizgilerinin dilde olduğunu gösterir. Eşitlik için başka hiçbir dizginin kabul edilmediği de gösterilmelidir. Burada bu da sağlanır: a'lar yalnız q_1'de okunur, b'ler başladıktan sonra a okunamaz, b sayısı a sayısını geçemez, q_3'e ise yalnız yığında \$ kalmışken geçilir.

> **Ders sonrası not:** Doğrulama: M_1, M_2 ve M_3 Python ile simüle edildi. Uzunluğu 12'ye kadar olan bütün $\{a, b\}$ dizgilerinde (8191 dizgi) sırasıyla $\{a^n b^n\}$, $\{vv^R\}$ ve a sayısı b sayısına eşit dizgiler dilleriyle tam uyum bulundu. $aaabbb$ ve $abba$ için üretilen betimlemeler bu nottaki tablo ve ağaçla aynıdır.

11. Bağlamdan Bağımsız Diller ve PDA'ların Denkliği

Bağlamdan bağımsız dilleri (context-free language, CFL) hatırlayalım. Dersin başında söylenen sonuç şimdi teorem olarak yazılır:

Teorem. Bağlamdan bağımsız gramerlerin ürettiği diller, PDA'ların kabul ettiği dillerle aynıdır.

Bu tür denkliklerde iki şey kanıtlanır:

- **Birinci yön:** Bağlamdan bağımsız diller, PDA'ların kabul ettiği dillerin alt kümesidir. Bu, herhangi bir CFG G'nin $L(G) = L(M)$ olan bir PDA M'ye dönüştürülebileceği demektir.
- **İkinci yön:** Bağlamdan bağımsız diller, PDA'ların kabul ettiği dillerin üst kümesidir. Bu, herhangi bir PDA M'nin $L(M) = L(G)$ olan bir CFG G'ye dönüştürülebileceği demektir.

Yani iki yönde de dönüştürme yapılır ve dönüştürülen yapı aynı dili verir. Bu dönüştürmeler sonraki derste yapılacaktır.

Dersin Toparlaması

Bu derste otomatlara yığın eklenerek PDA elde edildi. Bir geçiş artık bir giriş sembolü okur, yığından bir sembol çeker, yığına bir sembol iter. Bunların her biri null olabilir. PDA'lar deterministik değildir: Aynı anda birden fazla geçiş uygulanabiliyorsa makine çoğalır ve paralel hesaplamalardan biri kabul ederse PDA kabul eder. Kabul için girişin tümünün tüketilmesi ve son durumun kabul durumu olması gerekir, yığın içeriği önemsenmez.

M_1, M_2 ve M_3 üzerinden $a^n b^n$, vv^R ve sayı eşitliği dilleri incelendi. Anlık betimleme ile hesaplama yazıldı, $\vdash$ ve $\vdash^*$ üzerinden PDA'nın dili biçimsel olarak tanımlandı. Son olarak CFL'lerin PDA'ların kabul ettiği dillerle aynı olduğu teoremi söylendi. İki yöndeki dönüştürmeler sonraki derste yapılacaktır.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.